

《组成原理》指令集实验指导手册

缩放点积注意力中的 Query-Key 点积：

ARMv9-A 与 RISC-V 指令集对比实验



姓名：鲁昕宁

学号：2414015

班号：1012

1 实验目的

通过本实验，读者应完成以下目标：

1. 理解 Query-Key 点积在缩放点积注意力中的作用，并明确本实验只研究点积内核，不扩展到 softmax、多头或 Value 加权。
2. 掌握如何分别用 ARMv9-A / AArch64 与 RV64IM 汇编实现同一个点积循环，并在相同输入下验证功能一致性。
3. 掌握 ARMv9-A 在“不使用 SVE2（标量）”与“使用 SVE2（向量）”两种实现路径下的实验对比方法。
4. 学会在 WSL + QEMU User 环境下完成编译、运行、反汇编和批量计时。
5. 明确本实验只聚焦**单一代表性差异**：核心循环的代码密度（每轮静态指令数与循环体字节数）。
6. 理解“静态指令数更少”、“当前模拟平台上测得更快”、“真实平台上运行更快”并不是等同的概念。

1.1 面向授课对象的实验结论设定

本实验以“给其他同学上一次可复现实验课”为目标，预设希望读者最终独立得到以下结论：

1. ARMv9-A（AArch64）与 RV64IM 在实现同一乘加循环时，代码组织方式不同；
2. 在本实验工作负载中，ARMv9-A 核心循环的代码密度更高（循环更短）；
3. ARMv9-A 在当前 QEMU User 6.2.0 环境中，“使用 SVE2”不一定比“标量实现”更快；
4. 时间测量结果可作为补充观察，但不替代“单一差异点”的核心结论。

2 问题描述与应用场景

2.1 Q-K 点积

Transformer 中最常见的注意力打分步骤可以抽象为 Query 向量与 Key 向量的点积。若只保留最核心的一步，则可以写成：

$$score = \sum_{i=0}^{N-1} q_i \times k_i$$

这里的 *score* 可以看作缩放点积注意力中未归一化的相关性分数。真实模型中还会继续执行缩放、softmax 和对 *V* 的加权，但这些步骤会引入额外控制流与浮点计算，不利于把实验重点稳定地压缩到“点积循环如何用不同 ISA 表达”这一问题上。

本实验选取 Query-Key 点积作为实验对象，有三点原因。第一，它保留了注意力机制中最核心的乘加循环，具有明确的实际应用背景。第二，它同时包含顺序访存、乘法和累加，足够体现 ISA 在循环表达上的差异。第三，它可以被压缩为一个很小的汇编内核，便于使用反汇编工具准确锁定核心循环位置，并在扩大数据量后继续保持分析目标清晰。

因此，本实验关注的是同一个点积内核在 ARMv9-A 与 RISC-V 上的实现方式，以及在统一输入、统一 QEMU 环境下观察到的代码密度与运行时间结果。

2.2 ARMv9-A

ARM（Advanced RISC Machine）是一种基于精简指令集（RISC）的高效能、低功耗处理器架构，广泛应用于移动设备、嵌入式系统和服务器领域。ARMv8 开始引入 64 位处理能力，苹果的 A7 芯片率先采用了该版本。而 ARMv9-A 是 ARM 最新的应用处理器架构，被很多人们所熟悉的移动 SoC 采用，引入了很多新的关键特性，如可伸缩向量扩展（SVE2）、可伸缩矩阵扩展 2（SME2）、分支记录缓冲区扩展（BRBE）等。

2.3 RISC-V

RISC-V 是一种基于 RISC 架构的指令集架构，支持多种指令集扩展。RISC-V 最早只是为给 UCB 并行计算课程做一个教学工具而开发的，然而今天已经成为横跨汽车、数据中心、航天、AI 的全球计算基石。RISC-V 架构的不同部分可以以模块化的方式组织在一起，用户能够灵活选择不同的模块组合，来实现自己定制化设备的需要。

3 实验环境准备

3.1 软件环境

表 1: 实验环境

项目	说明
主机环境	Windows + WSL2
Linux 发行版	Ubuntu 22.04.5 LTS
模拟器	qemu-aarch64、qemu-riscv64
版本	QEMU User 6.2.0
ARM 目标	ARMv9-A, AArch64 state, 源码中使用 <code>.arch armv9-a</code>
RISC-V 目标	RV64IM, 显式关闭压缩指令扩展 C (<code>.option norvc</code>)
编译工具	gcc-aarch64-linux-gnu、gcc-riscv64-linux-gnu
反汇编工具	aarch64-linux-gnu-objdump、riscv64-linux-gnu-objdump
计时方式	Python <code>perf_counter_ns</code> , 每组 1 次热身 + 7 次实测取平均

3.2 推荐安装命令

若 WSL 中尚未安装交叉编译器，可在 Ubuntu 中执行以下命令：

```
apt-get update
apt-get install -y \
    qemu-user \
    gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu \
    gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

3.3 工具链兼容性说明

Ubuntu 22.04 默认提供的 `gcc-11` 对命令行参数 `-march=armv9-a` 的支持并不完整，但本实验的 ARM 源码中已经直接写入 `.arch armv9-a`，因此可以在不额外添加该命令行参数的情况下正确汇编和链接。由于实验内核只使用了 `ldr`、`madd`、`subs` 等基础 AArch64 指令，这样的处理不会影响本实验的代码密度和循环分析结论。

另外，作业说明中常写 `qemu-arm` 作为 ARM 运行示例；本实验代码为 AArch64（64 位）汇编，因此实际应使用 `qemu-aarch64`。两者并不冲突，本质上是目标 ISA 位宽不同导致的运行器选择差异。

4 算法设计与实验对象说明

4.1 统一输入模式

为保证两种 ISA 的实验完全可复现，本实验不再只使用单次长度为 8 的向量，而是采用“固定 8 元模式 + 重复扩展”的方式生成更长输入向量：

$$q_{\text{base}} = [1, 2, 3, 4, 5, 6, 7, 8], \quad k_{\text{base}} = [8, 7, 6, 5, 4, 3, 2, 1]$$

两者单次 8 元点积结果为：

$$1 \times 8 + 2 \times 7 + \cdots + 8 \times 1 = 120$$

若向量长度为 LEN ，且 LEN 是 8 的整数倍，则单轮点积结果为：

$$score_{\text{round}} = 120 \times \frac{LEN}{8}$$

若外层重复执行 $REPEAT$ 次，则程序最终校验值为：

$$score_{\text{total}} = score_{\text{round}} \times REPEAT$$

4.2 测试工作负载

本实验选择三组更大的测试数据量，全部由同一份汇编源码通过预处理宏 LEN 和 $REPEAT$ 编译得到：

表 2: 测试工作负载设置

向量长度 LEN	重复次数 $REPEAT$	RE- MAC 总次数	单轮点积结果	总校验值
256	20000	5,120,000	3,840	76,800,000
1024	20000	20,480,000	15,360	307,200,000
4096	20000	81,920,000	61,440	1,228,800,000

4.3 程序结构

两份汇编都采用相同的总体结构：

```
total_acc = 0
repeat_count = REPEAT

while repeat_count > 0:
    acc = dot(q, k, LEN)
    total_acc += acc
    repeat_count -= 1

if total_acc == EXPECTED_TOTAL:
    exit(0)
else:
    exit(1)
```

其中真正用于对比 ISA 差异的是 `dot_loop` 这一层内层循环。外层重复主要是为了让程序执行时间更稳定、便于测量。程序结束时采用“内部比对校验值，正确则退出码为 0，错误则退出码为 1”的方式完成正确性验证，因此不再受退出码 0-255 范围的限制。

5 实验步骤及预期结果

5.1 步骤一：运行一键基准脚本

目录中提供了 `benchmark_dot.py`，它会自动完成以下工作：

1. 按三组 `LEN` 和 `REPEAT` 参数分别编译 ARMv9-A 标量、ARMv9-A SVE2 与 RISC-V 程序；
2. 在 QEMU User 中执行 1 次热身与 7 次正式计时；
3. 使用 `objdump -d` 提取 ARM 标量与 RISC-V 的 `dot_loop`；
4. 输出 `benchmark_results.json` 与 `benchmark_results.md`。

在 WSL 的 `/mnt/d/1012/assign2` 目录中执行：

```
python3 benchmark_dot.py
```

5.2 读者在实验中应重点观察的指标

为了保证“单一代表性差异”不被其他因素干扰，建议读者按优先级观察：

1. 核心指标（必看）：`dot_loop` 的每轮静态指令条数、循环体字节数；
2. 辅助指标（选看）：同一工作负载下三程序（ARM 标量 / ARM SVE2 / RISC-V）在 QEMU User 的平均运行时间；
3. 有效性指标（必看）：程序退出码是否为 0（功能正确）。

5.3 步骤二：手动编译与运行示例

若只想手动验证其中一组，例如 `LEN=1024`、`REPEAT=20000`，可执行：

```
aarch64-linux-gnu-gcc -nostdlib -static -x assembler-with-cpp \  
-DLEN=1024 -DREPEAT=20000 \  
-o armv9_dot "1012-2414015-鲁昕宁-arm.s"  
  
riscv64-linux-gnu-gcc -nostdlib -static -x assembler-with-cpp \  
-DLEN=1024 -DREPEAT=20000 -march=rv64im -mabi=lp64 \  
-o riscv_dot "1012-2414015-鲁昕宁-riscv.s"  
  
aarch64-linux-gnu-gcc -static -O3 \  
-march=armv8.6-a+sve2 \  
-DLEN=1024 -DREPEAT=20000 \  
-o armv9_sve2_dot armv9_sve2_benchmark.c
```

```
qemu-aarch64 ./armv9_dot  
echo $?  
  
qemu-riscv64 ./riscv_dot
```

```
echo $?

qemu-aarch64 -cpu max ./armv9_sve2_dot
echo $?
```

若程序实现正确，三次 `echo $?` 的输出都应为 0。这说明 ARM 标量、ARM SVE2 与 RISC-V 版本都正确算出了总校验值。

5.4 步骤三：反汇编并定位核心循环

```
aarch64-linux-gnu-objdump -d armv9_dot > armv9_dot.dump
riscv64-linux-gnu-objdump -d riscv_dot > riscv_dot.dump
```

打开 `armv9_dot.dump` 与 `riscv_dot.dump` 后，搜索标签 `dot_loop` 和 `dot_loop_end`。两者之间的指令就是本实验需要统计的核心循环体。读者应重点观察：

- 每轮循环包含多少条指令；
- 哪些指令完成访存，哪些指令完成乘加，哪些指令完成循环控制；
- 某些操作是否被更强的单条指令融合。

5.5 步骤四：记录代码密度与时间结果

本实验的预期观察结果有三层：

1. ARMv9-A 版本的内层循环更短，静态指令数更少；
2. 在本机 WSL + QEMU User 6.2.0 的测量结果中，RISC-V 版本的平均模拟运行时间更短；
3. 在同样环境下，本次测量中 ARMv9 SVE2 版本平均时间高于 ARMv9 标量版本。

这两个结果并不矛盾。前者反映的是 ISA 在表达同一循环时的静态代码密度，后者反映的是当前模拟器实现与当前测试平台组合下的 wall-clock 时间。

5.6 预期发现对应的指令集原理

若读者按步骤完成实验，应能够将观察结果追溯到以下指令集层面的设计差异：

- ARMv9-A (AArch64) 可使用后索引访存（如 `ldr w4, [x0], #4`），在取数同时完成地址更新；
- ARMv9-A (AArch64) 可使用融合乘加指令 `madd`，将乘法与累加压缩到一条指令；
- RV64IM 在本实验约束下采用更细粒度的“访存/运算/更新”分解式指令序列，因此循环体更长。

6 核心代码片段与指令集差异分析

6.1 ARMv9-A (AArch64) 核心循环

```
dot_loop:
    ldr  w4, [x0], #4
    ldr  w5, [x1], #4
    madd x2, x4, x5, x2
    subs w3, w3, #1
    bne  dot_loop
```

6.2 RISC-V 核心循环

```
dot_loop:
    lw    t1, 0(a0)
    lw    t2, 0(a1)
    mul   t4, t1, t2
    add   s0, s0, t4
    addi  a0, a0, 4
    addi  a1, a1, 4
    addi  t0, t0, -1
    bnez  t0, dot_loop
```

6.3 核心循环代码密度对比

表 3: 核心循环代码密度对比

架构	每轮循环指令数	循环体字节数	主要原因
ARMv9-A (AArch64)	5	20 B	使用了后索引寻址的 <code>ldr</code> , 并用 <code>madd</code> 把乘法与累加融合为一条指令
RV64IM	8	32 B	访存、乘法、累加、地址更新和循环计数分别由独立指令完成

6.4 更大数据量下的时间测量结果

表 4: QEMU User 下的向量内积平均用时（三路对比）

LEN	REPEAT	MAC 总次数	ARM 标量	ARM SVE2	RISC-V	SVE2/标量	RISC-V/标量
256	20000	5,120,000	154.281 ms	170.006 ms	125.276 ms	1.102	0.812
1024	20000	20,480,000	199.683 ms	291.633 ms	167.765 ms	1.460	0.840
4096	20000	81,920,000	388.482 ms	830.929 ms	295.108 ms	2.139	0.760

6.5 结果解释

从静态循环可见, ARMv9-A (AArch64) 通过后索引寻址与 `madd` 指令, 把“取数并更新地址”以及“乘法并累加”压缩到更少的指令中, 因此在 **代码密度**这一指标上明显优于 RISC-V。

进一步观察 ARM 标量与 ARM SVE2 对比可见, 在当前 WSL + QEMU User 6.2.0 环境下, SVE2 版本并未体现更短的 wall-clock 时间, 且在更大数据量时差距扩大。这提示读者: **向量指令优势是否可见, 强依赖于运行平台是否真实支持并高效实现对应扩展**。在模拟器环境中, SVE2 可能受到翻译与仿真开销影响。

同时, 三组工作负载下测得的 wall-clock 时间仍表现为 RISC-V 更短。这说明: **静态指令数更少, 并不自动等价于当前模拟器中运行更快**。QEMU User 的翻译策略、不同后端的 helper 实现、宿主机执行路径以及系统噪声都会影响最终时间结果。

因此, 本实验得到两个可以同时成立的结论:

1. 若评价标准是核心循环的静态指令条数与循环体大小，则 ARMv9-A 的代码密度更高；
2. 若评价标准是本机 QEMU User 6.2.0 中的平均模拟执行时间，则本次测量中 RISC-V 更快；
3. 在当前模拟环境中，ARMv9 SVE2 版本不比 ARM 标量版本更快。

需要特别说明的是，第二个结论仅适用于当前模拟环境，不能直接外推为“真实 ARMv9 硬件一定慢于真实 RISC-V 硬件”。它更适合作为“代码密度结果和模拟运行时间结果可能分离”的一个实验观察。

7 实验流程与思路图



图 1: 更大数据量下的实验流程与思路图

8 实验评估方法

为判断读者是否真正完成并理解本实验，可从以下三方面进行评估：

表 5: 实验评估设计

评估维度	观察点	达标标准
实验操作	是否成功完成编译、QEMU 运行、反汇编和批量计时	两份程序均可运行，退出码为 0，并产出计时结果表
结果分析	是否能准确定位 <code>dot_loop</code> 并统计循环指令数	能给出 ARMv9 为 5 条、RISC-V 为 8 条的结论
知识掌握	是否能解释“代码密度”和“QEMU 时间结果”为什么可能不同	能说明模拟器实现、翻译策略与静态指令数并非一一对应

若读者只能“跑出时间”却不能说明 ARMv9 为什么循环更短，则说明实验操作完成了，但对 ISA 表达能力的理解仍不充分。反之，若读者既能分析循环体，又能说明为什么在 QEMU 中出现了与代码密度不同的时间结果，则说明实验目标已经达到。

9 附：源码目录与运行命令

```
1012-2414015-鲁昕宁-指令集实验设计.pdf
1012-2414015-鲁昕宁-arm.s
1012-2414015-鲁昕宁-riscv.s
armv9_sve2_benchmark.c
armv9_sve2_demo.c
armv9_sve2_demo.s
benchmark_dot.py
```

9.1 完整运行命令汇总

```
# 一键编译 + 运行 + 反汇编 + 计时
python3 benchmark_dot.py

# 手动验证一组参数
aarch64-linux-gnu-gcc -nostdlib -static -x assembler-with-cpp \
    -DLEN=1024 -DREPEAT=20000 \
    -o armv9_dot "1012-2414015-鲁昕宁-arm.s"
qemu-aarch64 ./armv9_dot
echo $?
aarch64-linux-gnu-objdump -d armv9_dot > armv9_dot.dump

riscv64-linux-gnu-gcc -nostdlib -static -x assembler-with-cpp \
    -DLEN=1024 -DREPEAT=20000 -march=rv64im -mabi=lp64 \
    -o riscv_dot "1012-2414015-鲁昕宁-riscv.s"
qemu-riscv64 ./riscv_dot
echo $?
```

```
riscv64-linux-gnu-objdump -d riscv_dot > riscv_dot.dump

# ARMv9 SVE2 版本 (C + SVE2 intrinsic)
aarch64-linux-gnu-gcc -static -O3 \
    -march=armv8.6-a+sve2 \
    -DLEN=1024 -DREPEAT=20000 \
    -o armv9_sve2_dot armv9_sve2_benchmark.c
qemu-aarch64 -cpu max ./armv9_sve2_dot
echo $?
```